

SwimData-IL

Integrating Israeli Swimming Association Results into a Relational Database

Data Management, BGU — Spring 2026 · Final Project Report

Asaf Belilus | github.com/Belilus/swimdata-il | Due 2026-07-27

1. Application overview

SwimData-IL ingests the official results of Israeli Swimming Association (ISA) competitions — published only as PDF — and turns them into a normalised relational database that answers questions the PDFs cannot: podiums per event, club medal tables, personal-best progressions (seed vs. final), age-group records, multi-event athletes, and disqualification analysis. A small console application runs these as parameterised SQL queries; the emphasis, per the brief, is on **data management, not UI**.

The database holds **two real championships**: *Arena Israel Summer 2026 Youth Championship (South)* (a two-source young-ages meet) and the *Arena Israel National Winter Championship 2025 (Youth & Seniors)* (downloaded and parsed from loglig) — together **4,770 swims** across **340 events** and **894 heats**, resolved to **1,311 swimmers** in **72 clubs**, with **zero orphan foreign keys**. Each meet alone (e.g. 1,701 swims / 74 events / 496 swimmers / 39 clubs for the first) is a complete dataset; the two together add a **cross-meet** identity dimension.

2. Data sources and description

All data is public, published by the ISA via isr.org.il/Competitions.ASP, which links to the timing provider **loglig.com**. Each competition exposes three PDFs:

Source	Language	What it contributes
Results	English	rank, heat, lane, swimmer, birth year, club, final time, FINA points, status (DNS/DQ/...)
Start list	Hebrew (RTL)	lane assignment, numeric club code , seed time (or NT), Hebrew swimmer & club names
Regulations (הוראות)	Hebrew	eligibility, age groups, entry limits — used to validate constraints (e.g. ≤ 4 events/swimmer, which the data confirms: $\max = 4$)

The PDFs are **fixed-column reports with no data layer**: naive text extraction (`pdftotext`) scrambles the columns because of RTL Hebrew headers, repeated page furniture, and club names that wrap across 2–3 lines. I therefore parse by **geometry** — reading each word with its `(x, y)` coordinates (`pdfplumber`), recovering the column bands empirically, and anchoring each swimmer on their year-of-birth token, then assigning every other word to its nearest anchor. This is the project's "raw-data → structured" step, exactly the pipeline the course opened with (*intro*: raw data → cleaning → structured DB → efficient queries).

3. Data-management challenges

The headline challenge is **multi-source bilingual entity resolution**, plus the missing-value and parsing challenges that fall out of real PDFs.

(a) Cross-language integration. The results (English) and start list (Hebrew) must be reconciled into one swimmer/club identity. My key observation: both PDFs share the tuple **(event-signature, heat, lane)**, and the start list carries a **language-neutral numeric club code**. Joining on `(distance, stroke, gender, age, heat, lane)` matches **1,672 /**

1,701 rows (98.3 %) one-to-one with zero ambiguity, and through that bridge each English name/club is tied to its Hebrew counterpart and to the federation code — deterministically, not by fuzzy transliteration.

(b) Club de-duplication / canonicalisation. The same club is spelled many ways in the results: the "Maccabi" prefix alone appears as *Macabbi / Maccabi / Maccabbi / Macabi*; casing is inconsistent (*hapoel bat-yam swimming club* vs *GREATER JERUSALEM SWIMMING CLUB*). Naively grouping by name would over-count clubs and even split one club in two. I key every club on its **federation code** and pick the **canonical name by statistical mode** (a window `ROW_NUMBER() ... ORDER BY count DESC`), recording every observed spelling in a `club_name_variant` audit table. Result: **72 raw spellings collapse to 39 clubs**. This step even *self-heals a data defect*: a `DNF` status token that leaked into one club cell created the bogus spelling *Maccabi Rishon Seals DNF* (seen once) — the mode correctly demotes it in favour of the 109-times-seen clean spelling. Three genuinely different Jerusalem clubs (*Hapoel, Maccabi, Greater Jerusalem*) are **not** merged, because their codes differ.

(c) Missing & special values. A result is **either** a time **or** a status — never both — enforced by a table `CHECK` . Seed times may be absent (`NT` = no time) and must stay `NULL` , *not* be treated as zero; the query for biggest improvement therefore excludes `NT` entries rather than crediting them an infinite drop. `DNS/DQ/DNF/NS` are normalised (`DQ` → `DSQ`) and their FINA rule reference (e.g. `SW 4.4`) is captured as its own column, enabling a "most common disqualification reason" query. **6 clubs have no English name anywhere in the source** (they print in Hebrew even in the English PDF) — a true missing value, handled by falling back to the Hebrew name (`COALESCE`).

4. Database design

The two PDFs are effectively **one wide, unnormalised relation** with heavy redundancy (club name and event attributes repeat on every swimmer row) and the classic insertion/update/deletion anomalies. I decompose to **BCNF**: every non-trivial functional dependency (`federation_code` → `club name` , `event_id` → `distance, stroke, gender, age` , `entry_id` → `result`) becomes its own table, so each fact is stored once.

```
+-----+ +-----+
| competition | | swimmer |
+-----+ +-----+
| 1 | *
| |
| * | *
+-----+ +-----+
| event | | club |
+-----+ +-----+
| 1 | 1
| |
| * | *
+-----+ +-----+
| heat | | club_name_variant |
+-----+ +-----+
| 1
|
| *
+-----+
| entry | <----+
+-----+ |
| 1 | 1
| |
| 1 | *
+-----+ +-----+
| result | | ref_stroke |
+-----+ +-----+
```

Key design decisions: `club.federation_code` is a natural unique key (the identity anchor for entity resolution); `entry` (seeded) and `result` (swum) are split 1-to-1 so an athlete who entered but did not start is modelled honestly; `ref_stroke` is a lookup table (FK from `event`) rather than a free-text column; times are stored as **integer milliseconds** (exact, sortable, index-friendly) and formatted only on display. Integrity is enforced with primary keys, seven foreign keys, `UNIQUE` constraints (`club.federation_code` , `heat(event_id, heat_no)` , `entry(heat_id, lane)` ,

`result.entry_id`), domain `CHECK s ( distance_m ∈ {25,...,1500} , gender , valid status codes, birth-year range)`, and the "time XOR status" rule. Indexes are chosen for the query workload (see §5).

5. Representative SQL queries

The full set is in `sql/04_queries.sql`; each is tagged with the course concept it exercises. Two examples:

Q1 — Podium per event (window functions, three-valued logic):

```
SELECT distance_m, stroke, gender, age_group, podium,
       first_name_en, last_name_en, club, final_time
FROM (
  SELECT f.*, RANK() OVER (PARTITION BY event_id ORDER BY final_time_ms) AS podium
  FROM v_fmt f
  WHERE final_time_ms IS NOT NULL -- DSQ/DNS must not take a medal
) t
WHERE podium <= 3;
```

Q3 — Club standings (join + conditional aggregation):

```
SELECT COALESCE(c.name_en, c.name_he) AS club,
       COUNT(*) FILTER (WHERE r.place = 1) AS gold,
       COUNT(*) FILTER (WHERE r.place = 2) AS silver,
       COUNT(*) FILTER (WHERE r.place = 3) AS bronze,
       COUNT(DISTINCT en.swimmer_id) AS swimmers
FROM result r
JOIN entry en ON en.entry_id = r.entry_id
JOIN swimmer sw ON sw.swimmer_id = en.swimmer_id
JOIN club c ON c.club_id = sw.club_id
GROUP BY 1 ORDER BY gold DESC;
```

Further queries in the repository: seed→final improvement (NULL-safe arithmetic on integrated columns; top drop **48.84 s**), `GROUP BY ... HAVING` (clubs with ≥ 10 swimmers), entity-resolution audit (`STRING_AGG` over `club_name_variant`), disqualification analysis (`GROUP BY dsq_rule`; SW 4.4 most common, 10×), and `INTERSECT` (swimmers in both Freestyle and Butterfly).

Indexing & plans (`sql/05_indexes_explain.sql`). Ties to the *Server* lectures: a composite B+-tree `swimmer(last_name, first_name, birth_year)` serves prefix name lookups (order matters); `result(final_time_ms)` supports the ranking/ `ORDER BY ... LIMIT` access path (sorted scan vs. full heap scan + sort); FK columns are indexed so the optimiser can pick **index-nested-loop** over block-nested-loop on the `result→entry→heat→event→swimmer` star join. `EXPLAIN / EXPLAIN ANALYZE` before and after `DROP INDEX` demonstrate the plan and cost change.

6. Repository and reproducibility

GitHub: <https://github.com/Belilus/swimdata-il> (source code, schema, SQL, parsers).

`bash build.sh` runs PDF → CSV → staging → normalised core → verification on DuckDB (zero server setup); the canonical DDL targets PostgreSQL. Verification asserts row counts and **zero orphan foreign keys**.